UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 12:

Archivos

Contenido

1. Introducción

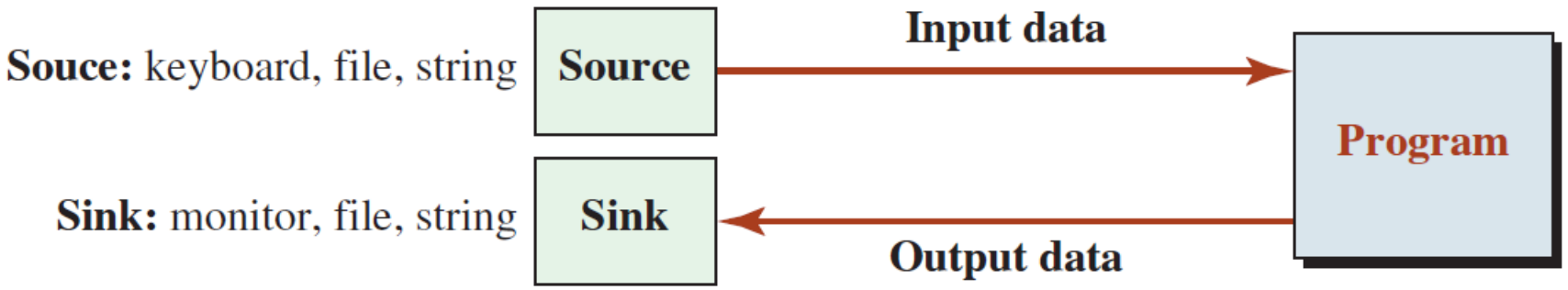
   Flujos de Entrada y Salida (I/O Streams)

   I/O texto vs binario , manejo de flujos (streams) mediante clases, flujos de consola, estados de stream

2. Archivos de texto
3. Modos de acceso
4. Creando un archivo secuencial.
5. Archivo binario
6. Archivos de acceso aleatorio

Introducción

- Cuando ejecutamos un programa, los **datos deben almacenarse en la memoria** para el procesamiento, estos datos provienen de una **fuente** externa y van a un **destino** externo.



- El almacenamiento de datos en la memoria es temporal. Los archivos se utilizan para la retención (permanente) de datos, que luego se pueden transferir físicamente a otra computadora y se pueden reutilizar.

Ejemplo: Cadenas como Fuente y Destino interno de datos

```
int main() {
    // Fuente (source): string
    string datoIngresado = "Hola Mundo";

    // Procesamiento: convertir a mayúsculas
    string datoProcesado = "";

    //ver https://www2.eii.uva.es/fund_inf/cpp/temas/9_vectores/range_for.html
    for (char c : datoIngresado) {//c es una copia
        datoProcesado += toupper(c);
    }

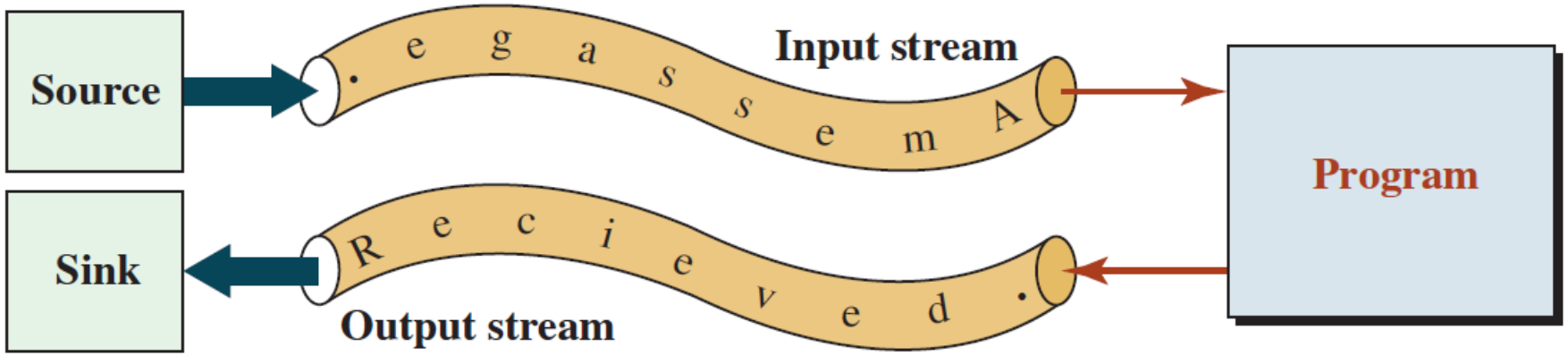
    // Destino(sink) : string
    string datoSalida = datoProcesado;

    // Mostrar el resultado en la consola (opcional, para ver el resultado)
    cout << "Datos procesados: " << datoSalida << endl;

    return 0;
}
```

Flujos (streams)

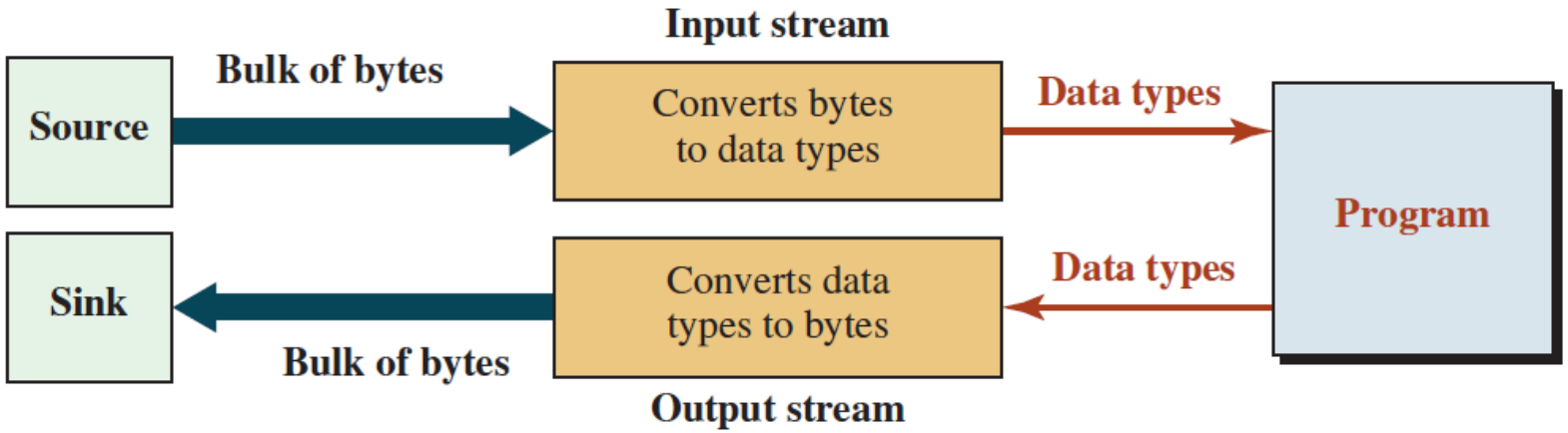
- Una fuente y un destino de datos no pueden conectarse directamente a un programa, es necesario un **mediador** entre ellos y el programa para controlar el flujo de datos y al mismo tiempo interpretar los datos al leer o escribir.



- Podemos interpretar el flujo como una secuencia de bytes que va desde una fuente o hacia un destino dentro de un programa

Datos vs Flujos

- Los datos enviados a un destino o desde una fuente (partes de la PC) tienen formato de texto o binario, sin embargo, se organizan en secuencia de bytes.



- El rol principal de los streams es convertir los bytes de datos que el programa recibe (envía) al tipo de dato que necesita (el receptor puede almacenar)

Ejemplo: flujo de entrada (teclado) / flujo de salida (consola)

```
int main() {
    // Fuente: teclado (Input stream)
    cout << "Introduce una cadena de texto: ";
    string datoIngresado; //incluir <string>
    getline(cin, datoIngresado); // Leer una línea completa de la entrada estándar

    // Procesamiento: Invertir la cadena de texto
    string datoProcesado = datoIngresado;
    reverse(datoProcesado.begin(), datoProcesado.end()); //incluir <algorithm>

    // Destino: pantalla (Output stream)
    cout << "Cadena invertida: " << datoProcesado << endl;
    return 0;
}
```

I/O Texto vs Binario

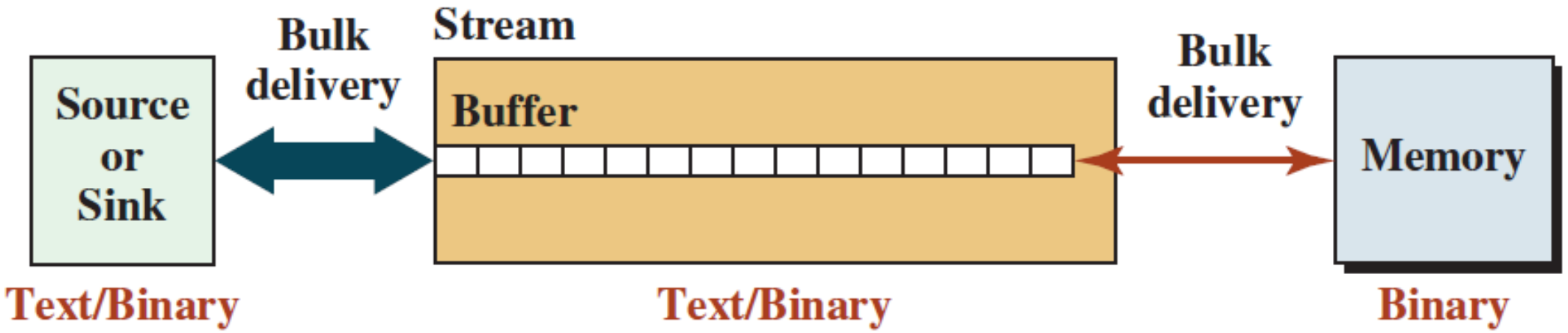
En el Ingreso y Salida de datos, están involucradas 3 ubicaciones:

1. **Memoria**, almacena datos en formato binario. Ejemplo sizeof(int) ==4

2. **fuente/destino**. También almacena datos en grupo de bytes, pero pueden ser texto o binario
   - ✓ El teclado/monitor como fuente/destino de datos solo acepta datos como texto.
   - ✓ Los archivos como fuente/destino de datos pueden manejar entrada/salida tanto de texto como binario. **Si queremos almacenar un objeto en un archivo el formato debe ser binario para preservar la estructura de datos del objeto**
   - ✓ Las cadenas como fuente/destino de datos tiene entrada/salida como texto

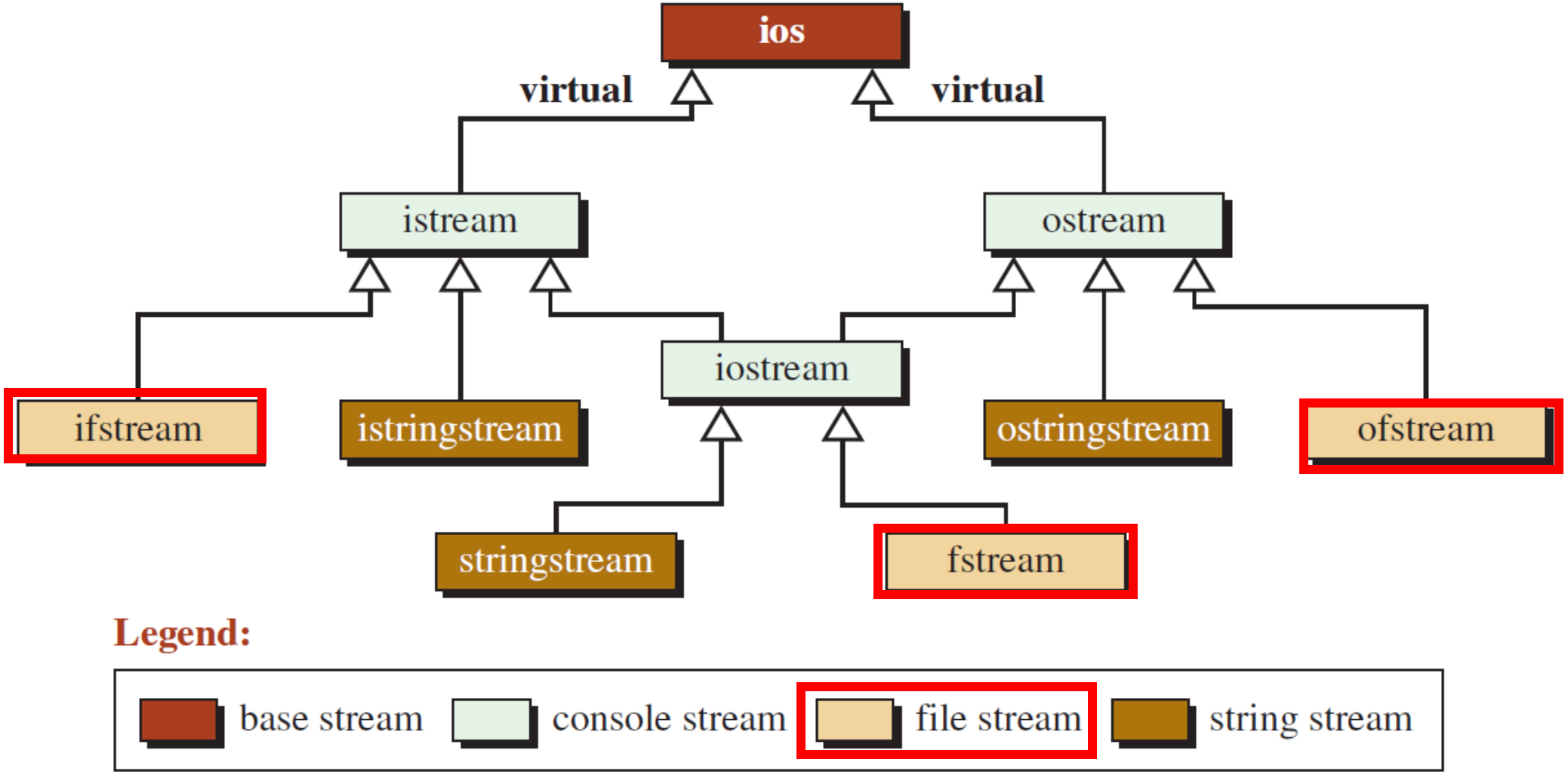
3. Búfer de flujo, Los flujos entregan o reciben datos en bloques, el búfer de flujo debe coincidir exactamente con el destino o la fuente a la que está conectado.

Por defecto, los streams operan en modo texto, para que operen en modo binario se debe crear una **instancia del objeto de flujo como binario**.



stream class

- Para manejar operaciones de entrada/salida, la biblioteca C++ define una jerarquía de clases. La clase ios no puede ser instanciada



Uso de stream

Para utilizar cualquier stream (flujo), debemos seguir los siguientes pasos:

1. Construir un objeto de stream apropiado (instanciar),
2. Conectar el objeto a la fuente o al destino,
3. Leer/Escribir los datos de/en la/el Fuente/Destino
4. Desconectar la fuente o el destino
5. Se destruye el stream

Características de los objetos de stream

Es importante recordar tres hechos sobre los objetos de stream instanciados:

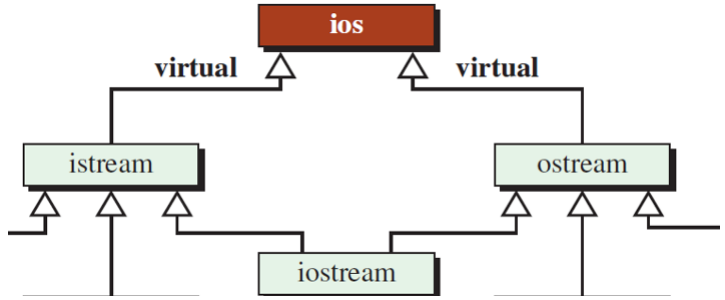
- No podemos pasar un objeto de cualquier clase de stream a una función por valor. El paso de estos objetos debe ser por referencia.

- No podemos devolver un objeto de cualquier clase de stream desde una función por valor; la devolución debe realizarse por referencia.

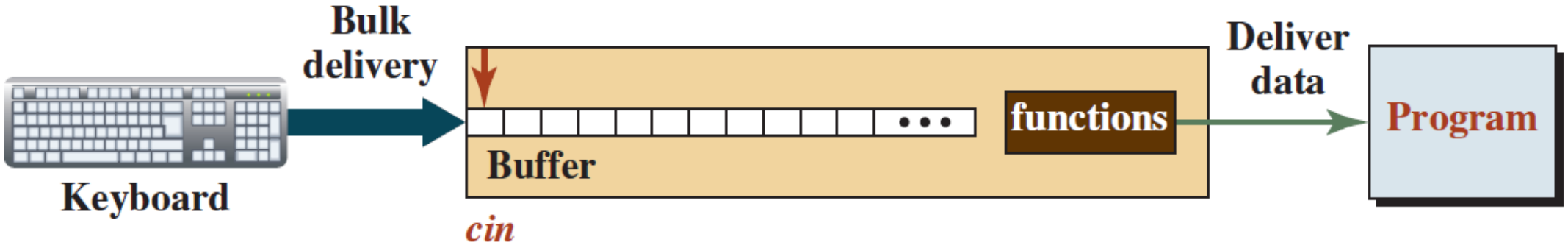
- No podemos pasar ni devolver un objeto de ninguna clase de stream usando un modificador constante; la naturaleza de estos objetos requiere cambios.

Ejemplo: streams como argumentos de funciones

Flujos de consola



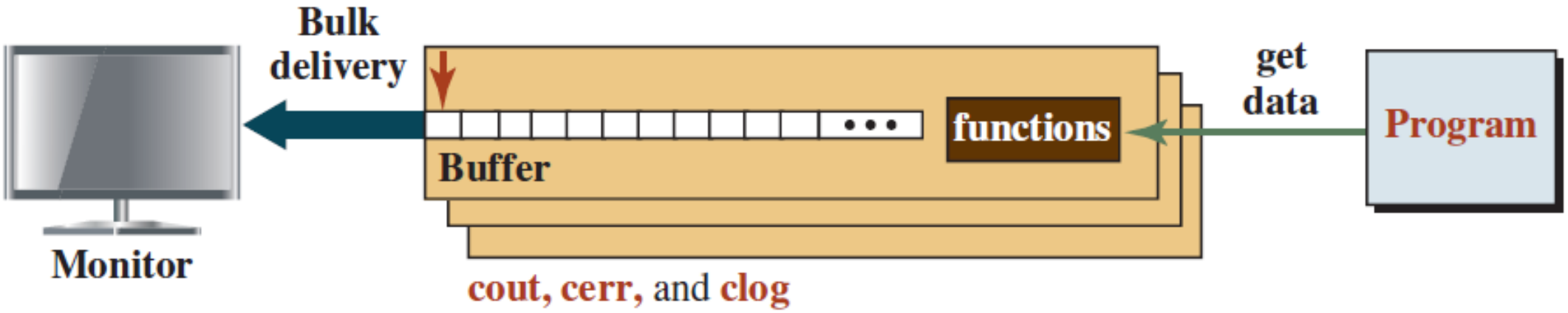
- Objeto de istream: **cin** se utiliza para lectura de datos, es creado por el sistema y se almacena en el archivo de encabezado <iostream>. El sistema también conecta el objeto cin al teclado.



- Cuando el programa sale de su ambíto, el objeto cin se destruye y se desconecta automáticamente del teclado.

Flujos de consola

- Objetos de ostream: **cout, cerr, clog** nos permite escribir datos desde nuestro programa al monitor, son creados por el sistema y conectados al monitor. Se almacenan en el archivo de encabezado <iostream>.



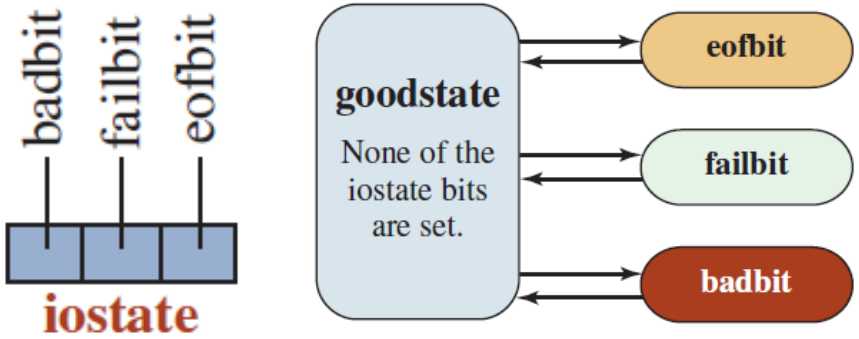
- Cuando nuestro programa sale del alcance, estos objetos se destruyen y se desconectan automáticamente del monitor. cout está vinculado al objeto cin. Tanto cerr como clog están diseñados para enviar errores a la consola.



Estados de flujo

- Un flujo debe realizar un seguimiento de su estado interno. Estos estados permiten gestionar los errores durante las operaciones de entrada o salida

miembros de datos



Constants	Input stream	Output stream
ios :: eofbit	No more characters to extract.	Not applicable.
ios :: failbit	An invalid read operation.	An invalid write operation.
ios :: badbit	Stream integrity is lost.	Stream integrity is lost.
ios :: goodbit	Everything is fine.	Everything is fine.

funciones miembros

Functions	Return values
bool eof ()	true if *eofbit* is set; false otherwise.
bool fail ()	true if *failbit* or *badbit* is set; false otherwise.
bool bad ()	true if *badbit* is set; false otherwise.
bool good ()	true if the *stream* is in good condition; false otherwise.
clear ();	It clears all three bits (sets to zero).
operator void* ();	It returns a pointer; interpreted *true* if not null.
const bool operator! ();	It returns true if *badbit* or *failbit* is set.

2. Archivos de texto

- Los archivos (ficheros) son la forma en la que C++ permite el acceso al disco. Se emplea los operadores << y >>

- Todos los procesos tienen abiertos por defecto los archivos de entrada, de salida y de salida de errores, que en C++ se corresponden con los objetos cin, cout , cerr y clog (que pertenece a la clase istream y ostream)

- **Archivos de Entrada**: El recorrido de un archivo de texto desconocido se hace con la lectura. Si en el directorio donde se ejecuta la aplicación hay un archivo llamado entrada.txt podemos verlo en pantalla utilizando **ifstream**

- **Archivos de Salida**: Se maneja con la clase **ofstream**

Ejemplo: Archivo de entrada

```cpp
#include <iostream>
#include <fstream> //manejo de archivos
using namespace std;

int main() {
    string s; //declarar o instanciar
    ifstream f; //("salida.txt"); // Abre un archivo para lectura
    f.open("salida.txt");
    if (f.is_open()) { // Verifica si el archivo se abrió correctamente
        getline(f, s); // Lee la primera línea del archivo

        while (!f.eof()) { // Mientras no se llegue al final del archivo
            cout << s << endl; // Imprime la línea en la consola
            getline(f, s); // Lee la siguiente línea del archivo
        }
    } else {
        // Imprime un mensaje de error si no se pudo abrir el archivo
        cout << "Error de apertura del archivo." << endl;
    }
    return 0;
}
```

Ejemplo: Archivo de salida

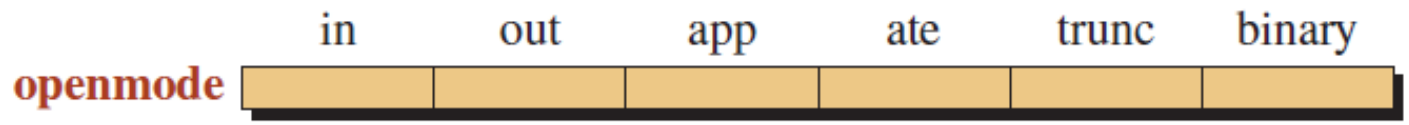
```cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Crea un objeto ofstream llamado 'f' y abre el archivo "salida.txt" para escribir
    ofstream f("salida.txt");

    if (f.is_open()) { // Verifica si la apertura del archivo fue exitosa
        f << "hola " << endl; // Escribe en el archivo
        f << 5 << endl;// Escribe en el archivo
    } else {
        // Imprime un mensaje de error si no se pudo abrir el archivo
        cerr << "Error de apertura del archivo." << endl;
    }
    return 0;
}
```

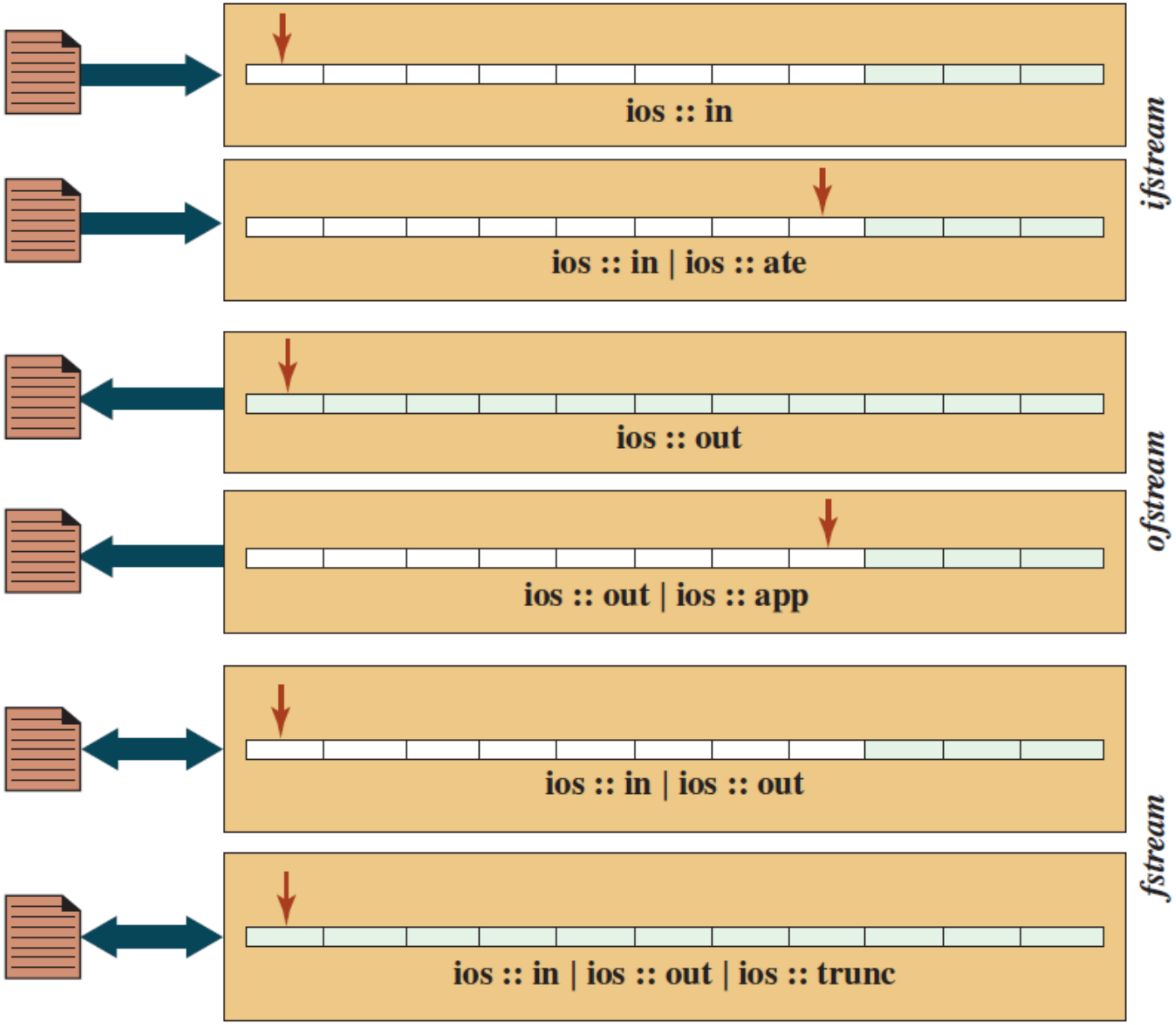
3. Modos de apertura

- El modo de apertura es un tipo definido en la clase ios y heredado por todas las clases en la jerarquía de clases de flujo. Se utilizan sólo en los flujos de archivos.



ios :: in	open for input
ios :: out	open for output; the contents are destroyed
ios :: app	open for output and write at the end (append)
ios :: ate	move to the end immediately after opening (at end)
ios :: trunc	truncate the file to zero length
ios :: binary	read and write in binary mode (default is text)

Combinaciones: modos de apertura



4. Creando un archivo secuencial

- C++ no impone ninguna estructura a un archivo (secuencia de bytes). Por tanto, un concepto como el de “registro” no existen un archivo C++.

- Debe estructurar archivos para cumplir con los requisitos de la aplicación.

- **Ejercicio 1**: Un banco requiere de un sistema de cuentas por cobrar para ayudar a administrar el dinero que sus clientes de crédito deben a una empresa. Para cada cliente, el programa obtiene el número de cuenta, el nombre y el saldo del cliente. Los datos obtenidos para cada cliente constituyen un registro para ese cliente. El número de cuenta sirve como registro clave; es decir, el programa crea y mantiene los registros del archivo en orden del número de cuenta.

Completar…

```
struct datosCliente {
    int numCuenta;
    char nombre[30];
    double saldo;
};
```

```
void agregarCliente(ofstream& archivoOut);
//usar modos de acceso adecuado
//limpia el buffer
```

```
void mostrarClientes(ifstream& archivoIn);
```

```
void buscarCliente(string& nombreArch, int numCuenta);
```

5. Archivos Binarios

- Son **similares a un conjunto de estructuras**, excepto que las estructuras están en un archivo de disco en lugar de la memoria. (ver Ejercicio 1)

- Podemos crear colecciones muy grandes de estructuras (limitadas únicamente por el espacio disponible en el disco), éstas son permanentes y siempre están disponibles. La única desventaja es la lentitud derivada del tiempo de acceso al disco.

- Características que lo distinguen de un archivo de texto:
  1. Puede saltar instantáneamente a cualquier estructura del archivo, lo que proporciona acceso aleatorio como en una matriz.
  2. Puede cambiar el contenido de una estructura en cualquier parte del archivo en cualquier momento.

Archivos Binarios

- Suelen tener tiempos de lectura y escritura más rápidos que los archivos de texto.

- Estos archivos admiten el concepto de **punteros a archivos**.
  - ✓ Cuando se abre el archivo, el puntero apunta al registro 0 (primer registro en el archivo)
  - ✓ **read** lee la estructura a la que apunta actualmente y mueve el puntero hacia abajo una estructura.
  - ✓ **write** escribe en la estructura a la que apunta actualmente y mueve el puntero hacia abajo una estructura.
  - ✓ **seek** mueve el puntero al registro solicitado.

ios :: beg	ios :: cur	ios :: end

direction (dir) values

Input	output
int tellg ();	int tellp ();
istream& seekg (int *pos*);	ostream& seekp (int *pos*);
istream& seekg (int *off*, ios :: dir);	ostream& seekp (int *off*, ios :: dir);

- La lectura o escritura usando el sistema de la consola solo se puede realizar de forma secuencial
- Podemos usar el acceso aleatorio cuando usamos los flujos de archivos. En acceso aleatorio, podemos leer datos desde cualquier ubicación y podemos escribir datos en cualquier ubicación, moviendo el marcador (puntero) de flujo al carácter correspondiente.
- Las clases istream y ostream proporcionan funciones miembros que nos permiten encontrar la ubicación del marcador y mover el marcador a la ubicación deseada, como se muestra en la Tabla precedente

Archivos binarios: Ejemplos

- Entrada y salida: Los archivos binarios permiten el posicionamiento, tanto del puntero de lectura como el de escritura. Estos punteros **sólo entienden de una posición en bytes en el archivo, y no de registros**.

- Para avanzar al registro n, será necesario emplear la fórmula: (n-1) * sizeof(Persona). Así, para el tercer registro:

```
f.seekp( 2 * sizeof( Persona ), ios::beg ); // escritura
f.seekg( 2 * sizeof( Persona ), ios::beg ); // lectura
```

Archivos binarios: Ejemplos

- Para saber en qué punto del archivo se encuentra uno de los punteros se utiliza los métodos:

```
f.tellg(); //puntero al leer datos
f.tellp(); //puntero al escribir datos
```

- Para abrir un archivo tanto para lectura y escritura, se debe utilizar la clase fstream, y posicionarse mediante el acceso directo para leer y escribir cuando sea preciso.

```
f.seekg( 2 * sizeof( Persona ), ios::beg );// ir al tercer registro y leerlo
f.read( (char *) &p, sizeof( Persona ) );

p.edad = 56;// cambiar el registro y guardarlo
f.seekp( 2 * sizeof( Persona ), ios::beg );
f.write( (char *) &p, sizeof( Persona ) );
```

Archivos binarios: Observaciones

- Los objetos de la clase string no se pueden utilizar, pues guardan la cadena en el heap, y no dentro de sí mismos. No puede utilizarse para archivos ningún objeto que no sea autocontenido.

- Un archivo binario generado con una máquina o un compilador distinto, del que generó el programa que leerá el archivo pueden hacer que el archivo sea ilegible.

- Por ejemplo, si crea un archivo binario en una PC con Windows, y compilador GNU gcc (Codeblocks) y pretende leer ese mismo archivo con un programa creado mediante la misma PC y Visual Studio, lo más probable es que la lectura no sea correcta, debido a formas distintas entre ambos compiladores de lidiar con la alineación en memoria.

Ejemplo: Archivo binario

Definiendo una estructura Persona

```
const int MaxNombre = 50;
const int MaxTelefono = 15;

struct Persona {
 char nombre[MaxNombre];
 int edad;
 char telefono[MaxTelefono];
};

void llenar(ostream & o, const Persona &p){
    o << nombre << ':' << edad << endl
      << "Tlf:" << telefono << ':' << endl
    ;
}
```

Guardando registro

```
Persona p1;
Persona p2;

strcpy( p1.nombre, "Baltasar" );
strcpy( p1.telefono, "988387028" );
p1.edad = 33;

strcpy( p2.nombre, "Pedro" );
strcpy( p2.telefono, "988387018" );
p1.edad = 33;

ofstream f( "datos.bin", ios::binary );
if ( f.is_open() ) {
    f.write( (char *) &p1, sizeof( Persona ) );
    f.write( (char *) &p2, sizeof( Persona ) );
} else cout << "Error de apertura de archivo."
<< endl;
```

...Ejemplo: Archivo binario

Realizando una lectura de cada registro

```
Persona p;
ifstream f( "datos.bin", ios::binary );

if ( f.is_open() ) {
    f.read( (char *) &p, sizeof( Persona ) );
    while( !f.eof() ) {
        llenar( cout, p );
        f.read( (char *) &p, sizeof( Persona ) );
    }
} else cout << "Error de apertura de archivo." << endl;
```

6. Archivos de acceso aleatorio

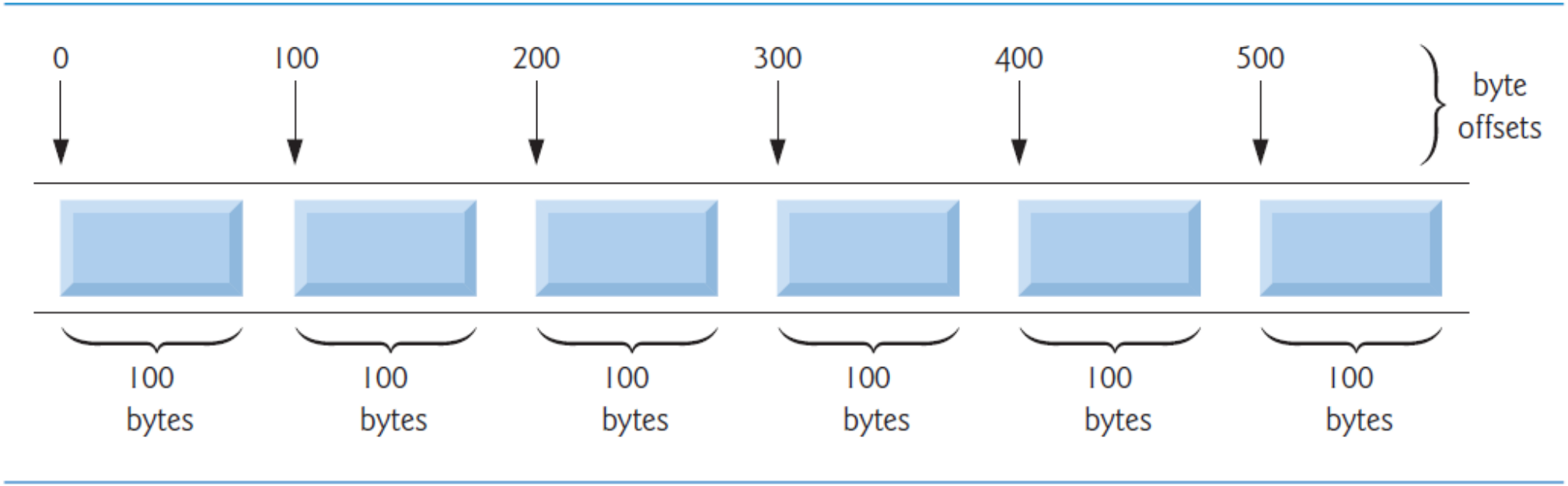
- Los archivos secuenciales no son apropiados para aplicaciones de acceso inmediato. Ejemplos: los sistemas de reservas de líneas aéreas, los sistemas bancarios, los sistemas de puntos de venta, etc. requieren un acceso rápido a datos específicos.

- Un banco puede tener miles de clientes, pero, cuando un cliente utiliza un cajero automático, el programa debe comprobar rápidamente que la cuenta de ese cliente tenga fondos suficientes.

- Este tipo de acceso instantáneo es posible con archivos de acceso aleatorio. Se puede acceder a los registros individuales de un archivo de acceso aleatorio directamente (y rápidamente) sin tener que buscar otros registros.

Archivos de acceso aleatorio

- Como C++ no impone estructura a un archivo, la aplicación que quiera utilizar archivos de acceso aleatorio debe crearlos.
- El método más sencillo es exigir que **todos los registros de un archivo tengan la misma longitud fija**, esto facilita que un programa calcule (en función del tamaño del registro y la clave del registro) la ubicación exacta de cualquier registro en relación con el comienzo del archivo.



Archivos de acceso aleatorio: Ventajas

- Los datos se pueden insertar en un archivo de acceso aleatorio sin destruir otros datos del archivo.

- Los datos almacenados anteriormente también se pueden actualizar o eliminar sin tener que volver a escribir todo el archivo.

- **Ejercicio 2:** Considere el Ejercicio 1 ahora utilice archivo binario con acceso aleatorio para que el sistema permita agregar o actualizar datos de un cliente, mostrar la información de un cliente, buscar un cliente usando su número de cuenta, eliminar un cliente de una posición específica. Su programa debe manejar las posiciones del archivo binario.

Creando un archivo de acceso aleatorio

- La función miembro de ostream **write** genera un número fijo de bytes, comenzando en una ubicación específica en la memoria, hasta la secuencia especificada.
  Cuando la secuencia está asociada con un archivo, la función escribe los datos en la ubicación del archivo especificada por el puntero de posición del archivo.

- La función miembro de istream **read** ingresa un número fijo de bytes del flujo especificado a un área en la memoria que comienza en una dirección especificada.
  Si la secuencia está asociada con un archivo, la función ingresa bytes en la ubicación del archivo especificada por el puntero de posición del archivo.

Escritura de bytes con la función write

- Utilizamos la declaración:

```
archSalida.write(reinterpret_cast<const char *>( &num),sizeof(num));
```

  que **siempre escribe la versión binaria** de los cuatro bytes del número entero num.

- La función **write** trata su primer argumento como un grupo de bytes al ver el objeto en la memoria como un carácter constante *, que es un puntero a un byte. A partir de esa ubicación, la función de escritura genera el número de bytes especificados por su segundo argumento.

- La función istream read se puede utilizar posteriormente para leer los cuatro bytes nuevamente

Conversión entre tipos de puntero: `reinterpret_cast<>`

- La mayoría de los punteros que pasamos a la función write como primer argumento no son de tipo const char *.
- Para generar objetos de otros tipos, debemos convertir los punteros a esos objetos para que escriban const char *; de lo contrario, el compilador no compilará las llamadas a la función write.
- C++ proporciona el operador `reinterpret_cast` para casos como este en los que un puntero de un tipo debe convertirse a un tipo de puntero no relacionado.
- Sin reinterpret_cast, la declaración write que genera el entero num no se compilará porque el compilador no permite que un puntero de tipo int * (el tipo devuelto por la expresión &num) se pase a una función que espera un argumento de tipo const char* .